# Technical report: Sokoto Rainfall Cover

**Capstone Project: Technical Implementation and Development Progress** · Chinemerem Judith Ugbo · Supervisor: Dr Aaron Izang

A weather-index insurance platform on Ethereum: automatic drought cover for smallholder farmers in Sokoto State, Nigeria. *Repository:* github.com/Phinatech/Capstone-project · *Live:* sokoto-cover.pages.dev · *Chain:* Ethereum Sepolia

## 1. Scope Alignment & Feature Validation

The approved scope proposed a blockchain-based weather-index insurance system in which rainfall is measured by multiple independent sources, policies are sold and settled by smart contracts without a claims process, and farmers, field agents and administrators manage cover through a web application. The delivered system matches that scope. One planned element (SMS delivery) is implemented but optional in configuration, and fiat settlement is deferred because it requires a licensed payment provider.

| Planned Feature | Intended Purpose | Current Status | Evidence |
|---|---|---|---|
| Smart contracts: policy sale, premium, payout reserve | Sell cover; guarantee payout trustlessly | **Completed** · M1 Contracts | WeatherIndexInsurance.sol; 56 tests; Policy #0 bought and paid out on Sepolia |
| Multi-source oracle with reputation-weighted aggregation | One source cannot decide a claim | **Completed** · M2 Oracle | OracleAggregator.sol; 3 live sources; simulator Fig. 07 |
| Automated oracle relayer and settlement keeper | Report rainfall and pay farmers unattended | **Completed** · M2 Oracle | relayer.ts, settlement.ts; Policy #0's live payout ran autonomously |
| Farmer portal (buy cover, | Self-service cover with a wallet | **Completed** · M3 Portals | /farmer/*; screenshots 02, 03, |

| Planned Feature | Intended Purpose | Current Status | Evidence |
|---|---|---|---|
| monitor, payouts) | | | 04 |
| Field-agent portal (sell cover, naira receipts) | Reach farmers without wallets | **Completed** · M3 Portals | /agent/*; receipts verified server-side against the chain |
| Admin console (policies, farmers and agents, analytics) | Operate and audit the scheme | **Completed** · M3 Portals | /admin/*; screenshots 05, 06 |
| Oracle research tools (consensus simulator, backtest) | Study and demonstrate the consensus design | **Completed** · M4 Research | /admin/backtest; 121 historical dekads (backtest-results.md) |
| Notifications (in-app, email codes, SMS/Telegram) | Tell farmers about purchases and payouts | **Completed (SMS optional)** · M4 Research | real event notifications; email codes; SMS behind SMS_PROVIDER |
| Testing and security audit | Validate and harden the system | **Completed** · M5 Validation | 348 tests; security-audit.md, fixes pinned by regressions |
| Deployment (public site and testnet) | Working product, end to end | **Completed** · M5 Validation | sokoto-cover.pages.dev + Render + Sepolia |
| Fiat settlement (naira off-ramp) | Pay farmers in naira | **Not implemented; justified** · M6 Future | needs a licensed provider; receipts record the naira leg; future work |

*Scope changes.* Nothing was dropped for convenience. The naira leg was designed as receipted off-chain settlement from the start, since a licensed off-ramp is beyond a student project. Deployment widened from a local chain to live Sepolia, which added wallet integration, RPC resilience and network-safety work.

## 2. Technical Product Demonstration

The walkthrough video (linked in the README) covers these in order.

**A. Core functionalities.** Policy purchase by an agent and by a farmer; the farmer dashboard and payouts ledger; the admin's oracle evaluation view; the consensus simulator corrupting one source; the backtest.

**B. Technical requirements.** Each screen is shown wired end to end: React/TypeScript to Express/TypeScript to Firebase to ethers.js to Solidity on Sepolia, fed by three live rainfall APIs.

**C. Testing with inputs.** The video shows a wrong premium, a closed window, insufficient pool capacity, a wrong wallet network, invalid codes and role escalation each refused safely, with a plain-language error.

**D. Testing strategies.** Unit and functional testing plus integration testing (section 3.3). **E. Performance and compatibility.** Measured latency and bundle sizes (section 3.4), desktop and mobile layouts, and a live cross-browser check.

## 3. Code Quality and Technical Implementation

### 3.1 OOP principles and architecture

The codebase is organised by layer (`blockchain/`, `backend/`, `frontend/`) with clear separation of concerns and no monolithic scripts.

*Encapsulation and abstraction.* Each store (`createUserStore`, `createSessionStore`, `createAgentStore`, `createAuthCodeStore`) hides its persistence behind one `StorePersistence` interface, so the application does not care whether records live in Firestore, a JSON file or memory. The policy layer hides its providers behind `usePolicies()` the same way.

*Polymorphism.* Persistence backends are interchangeable strategies (production, development, tests) chosen once at start-up. The consensus simulator runs three aggregation strategies behind one interface.

*Inheritance and reusable components.* `ErrorBoundary` extends `React.Component` once and protects every route. Typed contracts (`User`, `Policy`, `StorePersistence`) are implemented by interchangeable parts, and shared components (`PolicyBrowser`, `PageHeader`, `Avatar`) serve all three portals.

*Dependency injection and modularity.* `createAuthRouter({ users, sessions, authCodes })` receives its collaborators, so end-to-end tests exercise the real router against

isolated stores. Pure utilities (utils/oracle.ts, utils/currency.ts) and the Context tree layer shared behaviour without duplication.

## 3.2 Algorithms and problem-solving

*Reputation-weighted consensus.* Dekad rainfall is a weighted average over reporting sources, with weights from peer-agreement scores, so a corrupt source loses weight at once and keeps losing it. The simulator shows a corrupted feed moving the aggregate by 9% under reputation weights and 59% under equal weights.

*Out-of-sample backtest.* Weights are learned from inverse MSE on all years except the scored year, with 2,000-resample bootstrap confidence intervals, so the evaluation never fits the test data (backtest-results.md).

*Fixed-point time-series arithmetic.* Rainfall is stored as integer hundredths of millimetres and dekads as period ids (YYYYMMD). A window sums exactly the periods it contains, so settlement is exact and auditable.

*Concurrency-safe persistence.* Stores serialise writes through a promise chain and write to a temporary file before renaming it, which makes each update atomic. The relayer fetches sources with bounded concurrency (mapLimit) to respect free API tiers.

*Security primitives.* The system uses scrypt password hashing, constant-time API-key comparison, single-use attempt-capped verification codes and per-hour rate limits, each with regression tests.

## 3.3 Testing and validation

**Strategy 1: unit and functional testing.** Selected because the contract invariants are the product's fairness guarantees. The suite runs 56 Solidity tests (including a fuzz test over pool reserves), 237 backend unit tests and 42 frontend tests. It covers premium maths (10% of the payout, rounded up), reverting sales the pool cannot cover, payouts that happen once and only below the threshold, and every audit finding pinned by a named regression test. Result: 348 tests green in total (the 13 integration scenarios of Strategy 2 included), and the suite caught real defects (a GET-versus-POST resend route, a password stored in clear text at seed time, tests writing into production data), each fixed and pinned.

**Strategy 2: integration and end-to-end testing.** Selected because the guarantees span HTTP, storage, wallet and chain. Mocha drives the contracts through ethers.js as the application does (13 scenarios); auth routes run against a real

Express server with cookie jars; the deployed system was exercised live, and the first policy was bought and paid out autonomously. Result: flows verified in production. Failures traced to wallet infrastructure (stale RPC state, wrong network) led to hardening: gas pre-estimated on the app's own RPC, account switches followed, wrong-chain signing refused.

*Validation.* Invalid codes, wrong premiums, expired windows, role-escalation payloads and unreachable RPCs each produce a plain-language error rather than provider jargon.

## 3.4 Performance and compatibility

| Measurement | Result | Conditions |
|---|---|---|
| REST API latency | 0.44 to 0.82 s per read | Render free tier, warm |
| Frontend build | 8.3 s; 1.4 MB JS across lazy chunks | Vite production build, code split |
| Backend test suite | about 7 s, 237 tests | Node 22 |
| Layout | Responsive from desktop down to 390 px | captures 02, 09 |
| Browsers | Chrome, Edge, Firefox, Safari checked on the same build | walkthrough video, Scene 7 |
| Wallet | MetaMask on Sepolia, hardened against stale RPCs and wrong networks | EIP-1193 only; no browser-specific APIs |

The farmer side targets low-power hardware: the layout renders at 390 px, naira figures degrade gracefully when FX providers are down, and chain reads recover from RPC hiccups through retries.

## 3.5 Engineering quality

Commits are focused and imperative, naming is consistent, and comments explain decisions rather than restating code. Both workspaces compile under strict TypeScript. Supporting evidence lives in docs/screenshots/ (10 figures), docs/backtest-results.md, docs/security-audit.md, docs/debugging-journal.md and the live deployment.